P R O B E K L A U S U R Info I

Wintersemester 2008/09
Prof. Dr. Carsten Damm
20. Januar 2009
von 16:00+x bis 17:30+x Uhr
(Bearbeitungszeit: 90 Minuten)

Vorname:

Nachname:

Matrikelnummer:

Es sind **keine Hilfsmittel** (Skript, Taschenrechner etc.) erlaubt. Papier wird gestellt. Benutzen Sie nur die **ausgeteilten**, zusammengehefteten **Blätter** für Ihre Antworten. Schreiben Sie mit blauem/schwarzem Kugelschreiber, Füller etc..

Bleistift ist nicht erlaubt. Korrekturfelder | | **nicht beschriften**!

Zum **Bestehen** der Klausur sind **45** Punkte ausreichend.

Viel Erfolg!

	Max. Punkte	Schätzung für "4"
Aufgabe 1 (Auswahlaufgabe)	5+0	5
Aufgabe 2 (Java: eine einfache Klasse)	11+0	6
Aufgabe 3 (Datenstrukturen und Graphen)	13+0	6
Aufgabe 4 (Zahlendarstellung)	14+0	6
Aufgabe 5 (Algorithmen-Entwurf und Analyse)	16+2	7
Aufgabe 6 (Sortieren und Schleifeninvariante)	15+0	5
Aufgabe 7 (Rekursion und Iteration)	16+2	8
Summe	90+4	43

M U S T E R K L A U S U R Info I

Aufgabe 1 (Auswahlaufgabe [5+0 Punkte])

Nur jeweils eine Antwort ist richtig. Falsche Antworten werden innerhalb dieser Aufgabe verrechnet (Bsp.: a) und b) richtig, aber c), d) und e) falsch ergibt 0P).

Kreisen Sie die richtige Nummer ein. Mehr Kreise (oder unleserlich) ergeben 0P für die Teilaufgabe.

a) Überladen einer Methode

1. ist die Zuordnung mehrerer Semantiken zum gleichen Methodennamen (Auswahl erfolgt durch Aufrufkontext)
2. ist die Programmierung unübersichtlich vieler verschiedener Funktionalitäten innerhalb einer Methode
3. geschieht zum Beispiel bei der Neudefinition der Methode `Object.toString()` in einer abgeleiteten Klasse

Lösung 1.

b) imperative Programmierung ist gekennzeichnet durch

1. Kommandozeilensteuerung
2. Befehlsfolgen zur Zustandsänderung
3. Zerlegen von Problemen in Teilprobleme

Lösung 2.

c) Rekursion ist

1. das Rückverfolgen von Methodenaufrufen
2. das Aufrufen einer Methode in deren eigenem Methodenrumpf
3. das Rückführen einer Problemlösung auf zwei kleinere Probleme

Lösung 2.

d) Aus welchen drei Kontrollstrukturen lassen sich alle Algorithmen aufbauen?

1. Eingabe, Verarbeitung und Ausgabe
2. for-Schleife, while-Schleife und do-while-Schleife
3. Komposition, Alternative und while-Schleife

Lösung 3.

e) Algorithmen sind stets

1. in endlicher Zeit abarbeitbar
2. endlich beschreibbar
3. in asymptotischer Form analysierbar

Lösung 2.

Aufgabe 2 (Java: eine einfache Klasse [11+0 Punkte])
Geben Sie eine einfache Klasse `Modul` an, die man zur Verwaltung der Module in einem Bachelorstudiengang benutzen kann. Jedes Modul zeichnet sich durch seinen Namen, z.B. "Info I" oder "C-Kurs", sowie seine Leistungspunktezahl aus. Außerdem will man jederzeit die Anzahl angebotener Module ermitteln können, sowie die Gesamtanzahl der Leistungspunkte in diesen. *Benutzen Sie in Ihrem Quelltext die Bezeichner* ***`name`****,* ***`punkte`****,* ***`anzahl`*** *und* ***`gesamtPunkte`*** *für diese Dinge.* Die folgenden Operationen sollen möglich sein:

- Erzeugen einer Instanz dieser Klasse unter Angabe des Namens,
- Erzeugen einer Instanz dieser Klasse unter Angabe des Namens und der Leistungspunktezahl,
- Setzen und Abfragen der Punktzahl jeder einzelnen Instanz
- Es soll möglich sein, zu jeder Zeit abzufragen, wieviele Instanzen der Klasse bereits erzeugt wurden, und wieviele Leistungspunkte diese zusammen haben.

Lösung

```
public class Modul {                          // Punktzusammensetzung:
  String name;                                     // 0.5 Punkte
  long punkte;  // oder int oder sonstwas          // 0.5 Punkte
  static int anzahl = 0;                           // attr: 0.5 P,
                                                   // static: 0.5 P
  static long gesamtpunkte;                        // attr: 0.5 P,
                                                     static: 0.5 P
  public Modul(String t) { name = t; anzahl++; } // 1 Punkt
  public Modul(String t,long p) {                  // 1 Punkt
    name = t; anzahl++;
    punkte = p; gesamtpunkte = gesamtpunkte+p; }
  public void setPunkte(long p) {                  // 2 Punkte
    gesamtpunkte = gesamtpunkte - punkte + p;// vergessen/falsch=>nur 1P
    punkte = p; }
  public long getPunkte(long p) {                  // 1 Punkt
    return punkte; }
  public static int wieviele() { return anzahl;}
                                          // Meth: 1 P, static 0.5P
  public static long insgesamt() { return gesamtpunkte;}
                                          // Meth: 1P, static: 0.5P.
}
```

Aufgabe 3 (Datenstrukturen und Graphen [13+0 Punkte])

a) (3P) Auf einen anfangs leeren Stack werden `push`/`pop`-Operationen angewendet. Tragen Sie den Zustand des Stacks nach den angegebenen Aufrufen ein (erste Zeile ist komplett vorgegeben). Bitte schreiben Sie deutlich!

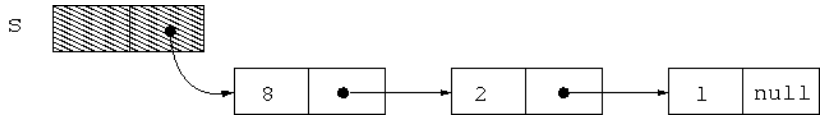
`push(1);push(2);push(8);`	1	2	8									
`push(5);push(3);pop();`												
`push(1);pop();pop();`												
`pop();push(7);push(1);`												

Lösung

`push(1);push(2);push(1);`	1	2	8									
`push(5);push(3);pop();`	1	2	8	5								
`push(1);pop();pop();`	1	2	8									
`pop();push(7);push(1);`	1	2	7	1								

Jede Abweichung gibt einen Punkt Abzug.

b) (6P)



Dies skizziert die Realisierung eines Stacks durch eine verkettete Liste mit Startknoten `s` (Zustand wie erste Tabellenzeile oben). Mögliche Befehle:

- *n* `= neu(`*i*`)` — Erzeugen von neuem Knoten namens *n* mit Inhalt *i*
- *x* `= inhalt(`*m*`)` — kopiere in *m* gespeicherte Zahl nach *x*
- `nachfolger(`*o*`) = `*p* — der Nachfolger von *o* ist nun *p*
  (oder auch: *k* `= nachfolger(`*z*`)` — Nachfolger von *z* in *k* speichern)

Geben Sie je eine Befehlsfolge an, um (1) bzw. (2) zu realisieren:
(1) `x = top(); pop();` (2) `push(5);`
Auf Reihenfolge achten! Auf Variablendeklarationen, Leerheitstest o.ä. brauchen Sie nicht achten!

Lösung

(1) `x = inhalt(nachfolger(s)); nachfolger(nachfolger(s)) = nachfolger(s);`

(2) `n = neu(5); nachfolger(n) = nachfolger(s); nachfolger(s) = n;`

Syntax nicht wichtig, solange das Gemeinte erkennbar ist. Je 3 Punkte. Für gut erkennbare Skizze (mit Reihenfolgekennzeichnung) können je nach Korrektheit auch Punkte gegeben werden — aber nicht die volle Punktzahl.

c) (4P) Welche Tiefe hat ein binärer Baum mit n Blättern mindestens? Geben Sie den Wert möglichst genau an und begründen Sie Ihre Antwort.

Lösung Ein binärer Baum hat in Tiefe 0 die Wurzel, in Tiefe 1 höchstens 2 Knoten, in Tiefe 2 höchstens 4 Knoten, ..., in Tiefe d höchstens 2^d Knoten. Die Tiefe eines solchen Baumes ist demnach die kleinste Zahl d mit $n \leq 2^d$. Anders ausgedrückt: $d = \lceil \log_2 n \rceil$.

$\log$ anstelle $\log_2$ anstreichen — kein Abzug

$\lfloor\ \rfloor$ anstelle $\lceil\ \rceil$, oder ganz ohne Klammer — 1 Punkt Abzug

$\log$, aber keine Begründung — 2 Punkte Abzug

Kein $\log$ — 2–3 Punkte Abzug (je nach Qualität der Begründung), sofern die Antwort nicht definitiv fehlerhaft ist (aber eben nicht genau), sonst 4 Punkte Abzug

Aufgabe 4 (Zahlendarstellung [14+0 Punkte])

a) (3P) Der hypothetische Datentyp `gint` stellt ganze Zahlen bekanntlich mit dem Zweierkomplement der Länge 4 dar (4-Bit-Zweierkomplement). Ergänzen Sie die folgenden Konstantendefinitionen der Wrapper-Klasse `Ginteger`:

```
public static final gint MAX_VALUE = ........;

public static final gint MIN_VALUE = ........;
```

Lösung 7 bzw. -8

(jeweils 1.5 P)

Richtiges Bitmuster, aber keine Zahl: 0.5P

b) (5P) Wir betrachten dazu noch den Datentyp `glint`: ganze Zahlen werden hier mit der 8-Bit-Zweierkomplement dargestellt. Ermitteln Sie die Bitmuster für 40 und -81.

Lösung (2P) $40 = (00101000)_{\overline{2}}$

(3P) $81 = (01010001)_{\overline{2}} \overset{\text{Komplement+1}}{\Rightarrow} -81 = (10101111)_{\overline{2}}$

[Bezeichnung unwichtig, Hauptsache Lösung+Rechenweg stimmen]

c) (6P) Ergänzen Sie diese Tabelle von `glint`-Werten.

Bitte deutlich schreiben, Nebenrechnung oder Probe nur auf Blatt rechts!

dezimal	binär (mit Zweierkompl.)	oktal	hexadezimal
13			
	$(10001011)_{\overline{2}}$		
		$(357)_8$	
			$(1E)_{16}$

Lösung

dezimal	binär (mit Zweierkompl.)	oktal	hexadezimal
13	$(00001101)_{\overline{2}}$	$(015)_8$	$(0D)_{16}$
-117	$(10001011)_{\overline{2}}$	$(213)_8$	$(8B)_{16}$
-15	$(11101111)_{\overline{2}}$	$(357)_8$	$(EF)_{16}$
30	$(00011110)_{\overline{2}}$	$(036)_8$	$(1E)_{16}$

pro richtigem Eintrag 1/2 Punkt

(Dinge wie z.B. $(15)_8$ anstelle $(015)_8$ sind *noch* OK)

Aufgabe 5 (Algorithmen-Entwurf und Analyse [16+2 Punkte])
Gegeben ist eine nur aus "+" oder "-" bestehende endliche Zeichenfolge, z.B. "`--+++--+++++-+-+-+-+--+`"
gespeichert in einem Feld `z[0]...z[k-1]`.

a) (8P + (2 Zusatzpunkte für (asymptotisch) optimalen Algorithmus)) Geben Sie einen Algorithmus an, der berechnet, wie lang die längste Teilfolge ist, die nur aus "+"-Zeichen besteht (im Beispiel: 5).

Der Algorithmus soll entweder in Java- oder Pseudocode- oder als Programmablaufplan oder als klar formulierter Text beschrieben werden.

Lösung optimal:

Initialisierung: $i = 0$, $n = 0$, $\max = 0$.
Schritt für Schritt nach rechts durchlaufen.

- Falls Zeichen an Stelle i ein "+" ist, $n = n+1$,
- Falls Zeichen an Stelle i ein "-" ist,
  falls $n > \max$, setze $\max = n$,
  setze $n = 0$.

nicht optimal: Schritt für Schritt nach rechts durchlaufen.

- wenn aktuelles Zeichen ='+', dann zähle durch while-Schleife, wieviele '+' danach kommen, Ergebnis + 1 in Hilfsfeld festhalten

Anschließend Maximum der Einträge in Hilfsfeld ermitteln

b) (5P) Geben Sie eine möglichst gute Abschätzung Ihres Algorithmus in $\mathcal{O}$-Notation an (mit kurzer Begründung).

Lösung [muss nicht so detailliert sein wie hier]

optimal in jeder Iteration der äußeren Schleife: höchstens 3 Vergleiche (Zeichen, Vergleich mit max, Abbruchbed.), höchstens eine arithmetische Operation, höchstens 2 Zuweisungen: d.h. jede Iteration kostet nur konstante Zeit. Da k Iterationen folgt Laufzeit $\mathcal{O}(k)$

nicht optimal erste äußere Schleife: k Iterationen, innere Schleifen bis zu $k-1$ Iterationen (man kann es auch komplizierter durch arithmetische Reihe angeben), in jeder inneren Iteration nur konstant viele Elementaroperationen, folglich: Laufzeit für erste äußere Schleife kann nur durch $\mathcal{O}(k^2)$ abgeschätzt werden, Maximumbestimmung $\mathcal{O}(k)$ (wenn man es richtig macht), d.h. insgesamt $\mathcal{O}(k)$

c) (3P) Ein Algorithmus zur Lösung von a) liest beginnend bei Index 0 fortlaufend die Feldeinträge (und stellt interne Berechnungen beliebiger Art an). Das Lesen wird aber sofort unterbrochen, wenn die Antwort feststeht. Der Algorithmus kann die Feldlänge im vorhinein ermitteln.

Geben Sie Zeichenketten (1) und (2) der Länge 8 wie folgt an: (1) Es muss bis zum Ende gelesen werden. (2) Es muss *nicht* bis zum Ende gelesen werden und das Resultat ist 4. Begründen Sie jeweils kurz.

Lösung

(1) ++++++++ — hört er vorher auf, könnte ja noch ein + kommen, würde falsche Antwort geben

(2) ++++---- — nach 4 + eine Unterbrechung: da Länge bekannt, kann nichts Längeres kommen, also Antwort 4 steht fest (auch -++++-+- etc. wäre richtig)

je 0.5 + 1 Punkte für Beispiel + Begründung

Aufgabe 6 (Sortieren und Schleifeninvariante [15+0 Punkte])
Wir betrachten folgenden elementaren Sortieralgorithmus

```
30    public static void selectionSort(int[] feld) {
31        int maxpos, tmp;
32        for (int i=feld.length; i>1; i--) {
33            // bestimme Position des Maximums im Intervall [0,...,i-1]
34            maxpos = 0; // Position des Maximums
35            for (int j=0; j<i; j++)
36                if ( feld[j] > feld[maxpos])
37                    maxpos = j;
38            // tausche feld[i-1] und feld[maxpos]:
39            tmp = feld[i-1]; feld[i-1] = feld[maxpos]; feld[maxpos] = tmp;
40        }
41    }
```

a) (4P) Wenden Sie den Algorithmus an auf
`int[] test = {1, 6, 9, 8, 2, 3, 4, 5, 7};`
und ergänzen Sie die Tabelle um die Feldinhalte *nach* Iterationen 1, 2 und 3 der *äußeren* `for`*-Schleife* (Zeilen 32-40).

Start	1	6	9	8	2	3	4	5	7
nach 1. Iteration									
nach 2. Iteration									
nach 3. Iteration									

Lösung

167823459
167523489
164523789

b) (5P) Geben Sie (mit Begründung) eine möglichst gute asymptotische Abschätzung der Laufzeit des Algorithmus auf Eingaben der Länge n an

Lösung Äußere Schleife besteht im Wesentlichen aus konstant vielen Zuweisungen und einer inneren Schleife der Länge i (Laufvariable der äußeren Schleife), die wiederum nur aus Vergleichen und Zuweisungen besteht. Deswegen ist deren Laufzeit proportional zu i und insgesamt erhalten wir die Laufzeitschranke $\mathcal{O}(n + (n-1) + \ldots + 2) = \mathcal{O}(n^2)$.

[Ich denke, die Formel für die arithemtische Reihe ist hinlänglich bekannt und braucht hier nicht bewiesen werden.]

c) (6P) Es gilt folgende Aussage: Nach Ausführung der inneren Schleife (Zeilen 35–37) gilt `feld[maxpos]` enthält das Maximum der Einträge im Teilfeld `feld[0..i-1]`.

Begründen Sie damit, dass das Feld nach `feld.length` − 1 Iterationen der *äußeren Schleife* (Zeilen 32–40) sortiert ist. Das formale Konzept der Schleifeninvariante können, müssen Sie aber nicht benutzen.

Lösung

[großzügig korrigieren]

informal:

Zeile 39 sorgt dafür, dass das Maximum in `feld[0..i-1]` mit `feld[i-1]` die Positionen tauscht. Nach der ersten Iteration der äußeren Schleife haben wir also das Feldmaximum am Ende und stehen dann vor der gleichen Situation wie vorher — jedoch eingeschränkt auf das Teilfeld `feld[0..feld.length-2]`. Dessen Maximum rückt nun an dessen Ende, usw. Dies wird `feld.length-1` mal durchgeführt — wir erhalten schließlich ein Feld, bei dem die `feld.length-1` größten Einträge sortiert am Ende stehen und der Verbleibende in Position `0`. Da die letzten Einträge alle nicht kleiner sind, ist das ganze Feld sortiert.

formal $\mathcal{I}$ = „das Teilfeld T =`feld[i..feld.length-1]` ist aufsteigend sortiert und jeder Wert in `feld[0..i-1]` ist kleiner gleich jedem Wert in T“

$\mathcal{I}$ ist wahr vor der ersten Iteration ($i =$ `feld.length`), denn in diesem Fall haben wir ein leeres Feld T: nirgends ist die Ordnung verletzt und es gibt auch keine Einträge in T, die kleiner als ein Eintrag das Anfangsstücks sein könnten.

Sei $\mathcal{I}$ wahr vor Iteration $2 \leq i \leq$ `feld.length`. Das heißt `feld`$[$`i..feld.length` $- 1]$ ist aufsteigend sortiert und alle Einträge am Anfang sind kleiner gleich `feld[i]`. Da durch die innere Schleife das Maximum im Anfangsstück bestimmt wird und in Zeile 39 den Platz mit `feld[i-1]` tauscht, gilt wieder $\mathcal{I}$.

Nach der letzen Iteration gilt $\mathcal{I}$ also auch.

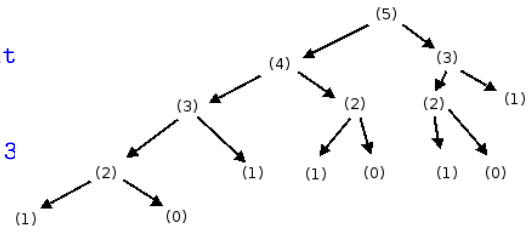
Die letzte Iteration beginnt mit `i==2` und endet mit `i==1`. $\mathcal{I}$ bedeutet somit: die letzten `feld.length-1` Einträge sind aufsteigend sortiert und der erste Eintrag (Index `0`) ist kleiner gleich allen Einträgen im Endstück. Somit ist das Feld aufsteigend sortiert.

Aufgabe 7 (Rekursion und Iteration [16+2 Punkte])
Pseudocode ist ausreichend für diese Aufgaben. Die Zahlenfolge $a_0, a_1, a_2, \ldots$ ist definiert durch: $a_0 = 0, a_1 = 1$ und $a_n = -2a_{n-1} + 3a_{n-2}$ für $n > 2$.

a) (6P) Formulieren Sie einen *rekursiven* Algorithmus, der bei Eingabe n (natürliche Zahl) den Wert a_n zurückgibt (nicht druckt!). Zeichnen Sie den Aufrufbaum (Rekursionsbaum) bei Aufruf mit der Eingabe $n = 5$.

Lösung

```
3       private static int aRek(int
4           if ( n == 0) return 0;
5           if ( n == 1) return 1;
6           return -2*aRek(n-1) + 3
7       }
```



4P für Alg, 2P für Baum

b) (6P) Formulieren Sie einen *iterativen*[1] Algorithmus, der bei Eingabe n (natürliche Zahl) den Wert a_n zurückgibt (nicht druckt!).

Lösung

```
8       private static int aIte1(int n) {
9           int[] a = new int[n+1];
10          a[0] = 0;
11          if (n > 0) a[1] = 1;
12          if (n > 1) {
13              for (int i = 2; i <= n; i++)
14                  a[i]=-2*a[i-1] + 3*a[i-2];
15          }
16          return a[n];
17      }

18      private static int aIte2(int n) {
19          int akt = 0, vor=1, vorvor=0;
20          if ( n == 0) return 0;
21          if ( n == 1) return 1;
22          for (int i = 0; i <= n-2; i++) {
23              akt = -2*vor + 3*vorvor;
24              vorvor = vor;
25              vor = akt;
26          }
27          return akt;
28      }
```

[1] Zwei Möglichkeiten: (1) Feld anlegen, das nach und nach mit den Werten a_0, a_1, a_2, ... gefüllt wird oder (2) lediglich die letzten Werte speichern, daraus den nächsten berechnen und geeignet umspeichern usw.

c) (4P + 2 Zusatzpunkte für Beweis der Formel) Für alle n gilt $a_n = \frac{1-(-3)^n}{4}$. Formulieren Sie aufgrund dessen einen Algorithmus, der bei Eingabe n (natürliche Zahl), den Wert a_n zurückgibt (nicht druckt!). Potenzieren darf dabei *nicht* benutzt werden.

Lösung

Die Aussage gilt offenbar für $n = 0$ und $n = 1$. Gelte die Aussage für $n \geq 1$. Dann folgt die Aussage für $n + 1$ aus der Gleichung

$$\frac{1-(-3)^{n+1}}{4} = -2\frac{1-(-3)^n}{4} + 3\frac{1-(-3)^{n-1}}{4},$$

die wiederum äquivalent ist zu der wahren Gleichung

$$1-(-3)^{n+1} = 1 + 2(-3)^n - 3(-3)^{n-1}.$$

```
29      private static int aImm(int n) {
30          int p = 1;
31          for (int i = 0; i < n; i++)
32              p*=(-3);
33          return (1-p)/4;
34      }
```